LAB SHEET 8

Performance Evaluation: Throughput, Jitter, and Loss vs. Distance

Week	8
Duration	2 hours
Deliverable	measure.py, plot_results.py, results.json, performance_plot.png, report (PDF), pushed to your netprog-2026 GitHub repository

Learning Objectives

By the end of this lab, you should be able to:

1. Explain the four core performance metrics: throughput, delay (latency), jitter, and packet loss.

2. Use iperf3 in UDP mode to directly measure throughput, jitter, and packet loss between two Mininet-WiFi stations.

3. Parse iperf3's JSON output programmatically from a Python script.

4. Automate a parameter sweep (distance from the access point) and collect performance data across multiple measurement runs.

5. Visualize collected performance data using matplotlib and interpret the resulting trend.

Background

Four metrics recur throughout network performance work, and today's lab measures all four in one pass:

Throughput	The rate of data successfully delivered, typically in bits/sec — what iperf3 reports as bits_per_second.
Delay / latency	The time for a packet to travel from sender to receiver, usually estimated from ping RTT / 2, or measured one-way in more advanced setups.
Jitter	The variation in delay between consecutive packets — iperf3's UDP mode reports this directly as jitter_ms, since inter-packet timing is exactly what matters for real-time traffic like voice/video.
Packet loss	The percentage of packets sent that never arrived — reported directly by iperf3's UDP mode as lost_percent, since UDP (unlike TCP) does not retransmit or hide losses from the measurement.

iperf3's TCP mode (the default) is excellent for pure throughput, but TCP's own retransmission and congestion control hide packet loss and jitter from the measurement entirely. UDP mode (-u), by contrast, sends a fixed target rate (-b) and reports exactly how much of it arrived, how unevenly spaced it was (jitter), and how much never showed up (loss) — which is why today's lab uses -u

throughout. Passing -J requests machine-readable JSON output instead of iperf3's human-readable text, so your script can parse exact numeric fields with Python's json module instead of scraping text.

> **Reading iperf3's UDP JSON output**
>
> For a UDP test, the field you want sits at data["end"]["sum"] in the client's JSON output, with sub-fields bits_per_second, jitter_ms, and lost_percent. (TCP mode uses a different structure — sum_sent / sum_received — which is one more reason today's lab standardizes on UDP.)

Prerequisites

- Lab 6 completed (comfort with addStation(), addAccessPoint(), and the range parameter)
- Comfort with Python dictionaries, the json module, and basic file I/O (from Labs 2-3).

Software & Materials

• Mininet-WiFi (installed in Lab 1), Python 3.8+
• iperf3 (not the older iperf — JSON output requires iperf3 specifically)
• matplotlib (pip install matplotlib inside your netprog-env virtual environment from Lab 1)

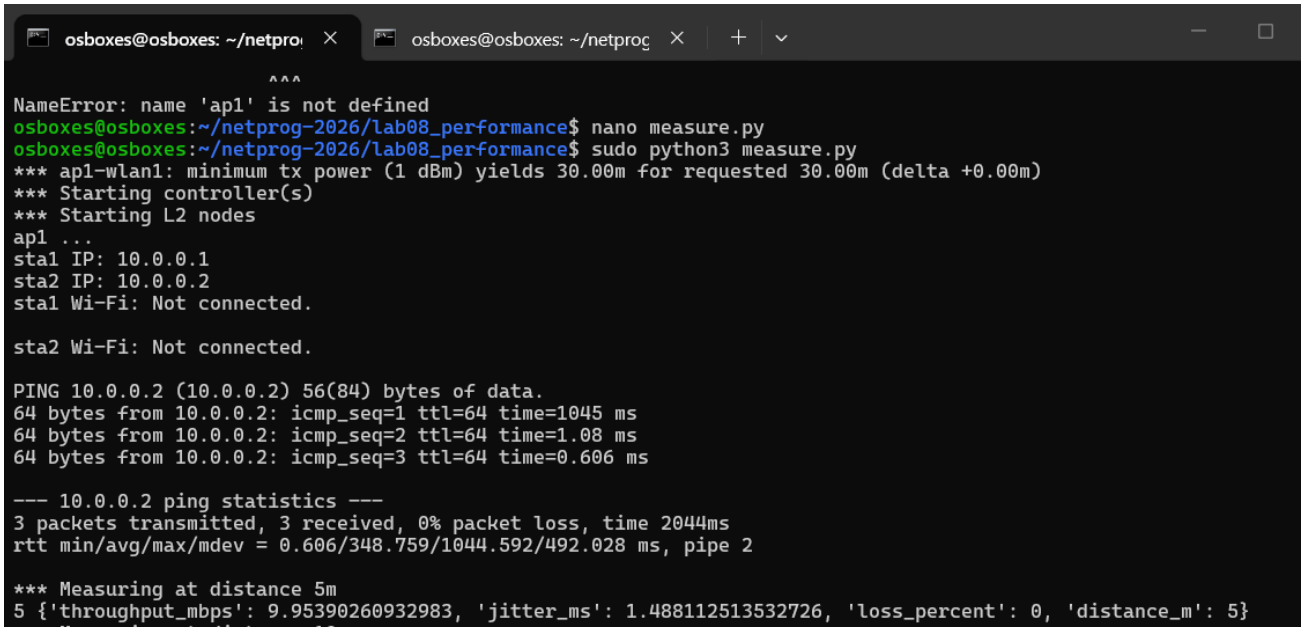
Lab Tasks

Complete the following tasks and record evidence (terminal output / screenshots) for each in your report.

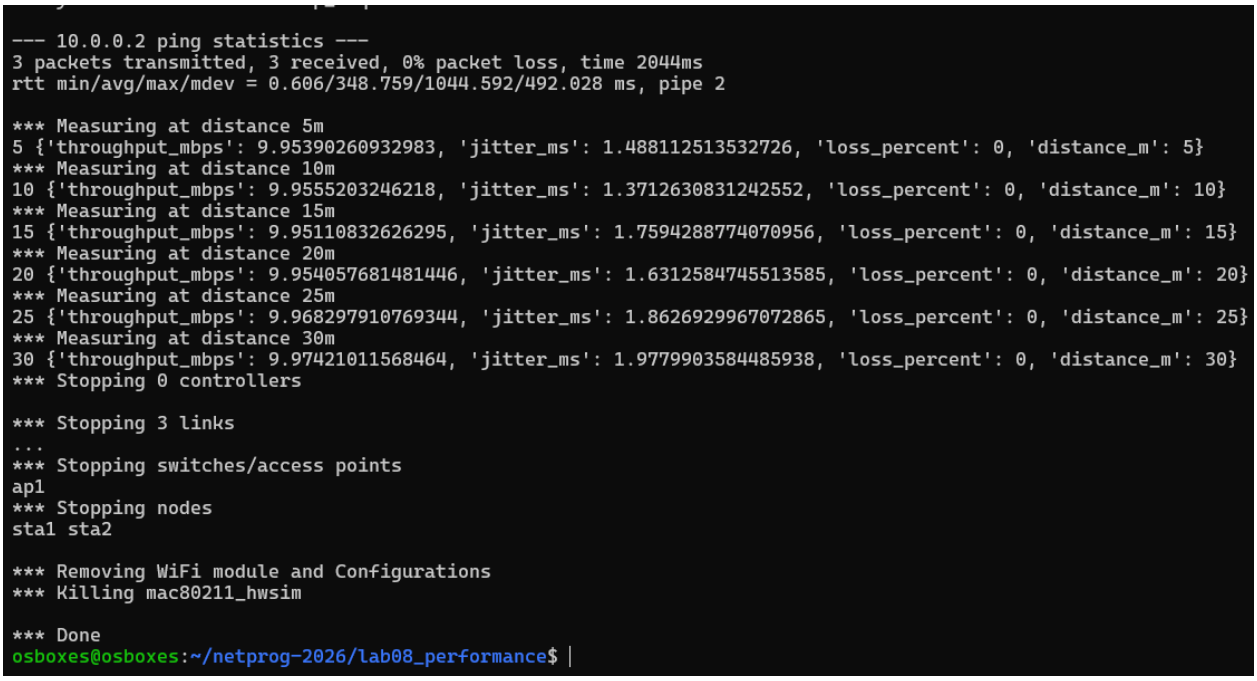
Task 1: Screenshot of Part B's manual iperf3 JSON output, with bits_per_second, jitter_ms, and lost_percent visibly located inside ["end"]["sum"].



Task 2: Screenshot of run_iperf3_udp() being called once and printing a result dictionary for a single distance.



Task 3: Screenshot of Part D's full sweep output (all six distances printed), plus the contents of the resulting results.json.

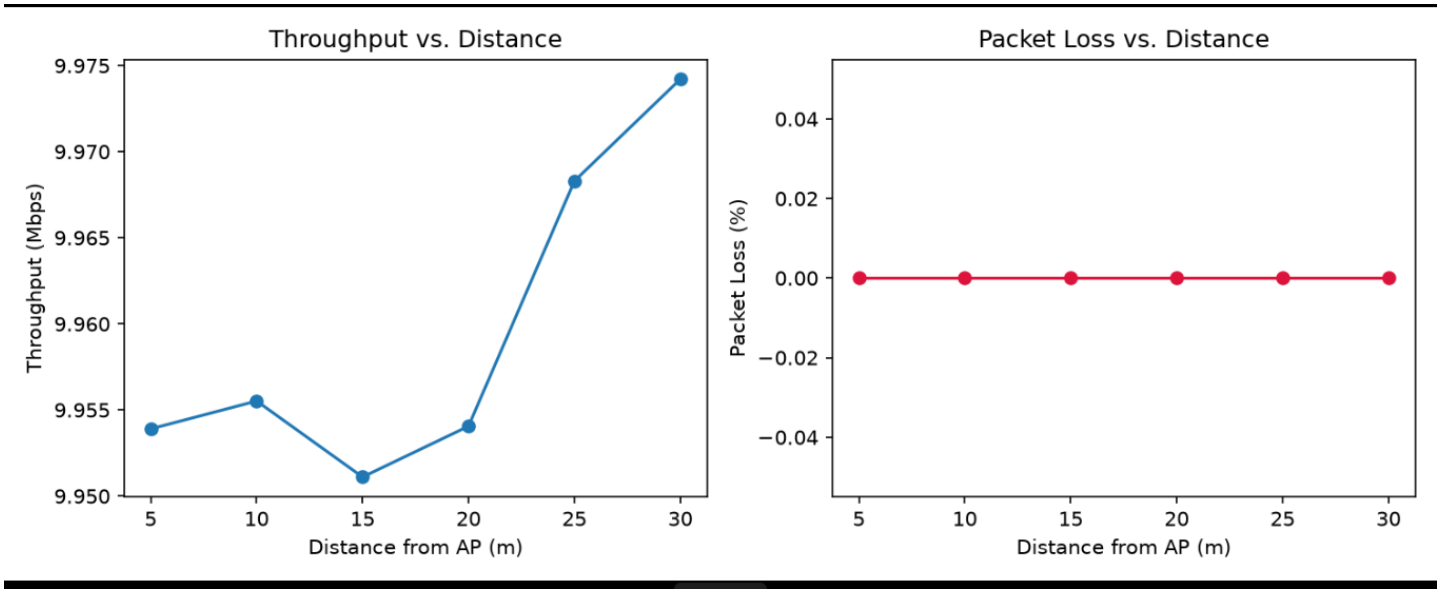


The result of result.json:

```
osboxes@osboxes:~/netprog-2026/lab08_performance$ cat results.json
[
  {
    "throughput_mbps": 9.95390260932983,
    "jitter_ms": 1.488112513532726,
    "loss_percent": 0,
    "distance_m": 5
  },
  {
    "throughput_mbps": 9.9555203246218,
    "jitter_ms": 1.3712630831242552,
    "loss_percent": 0,
    "distance_m": 10
  },
  {
    "throughput_mbps": 9.95110832626295,
    "jitter_ms": 1.7594288774070956,
    "loss_percent": 0,
    "distance_m": 15
  },
  {
    "throughput_mbps": 9.954057681481446,
    "jitter_ms": 1.6312584745513585,
    "loss_percent": 0,
    "distance_m": 20
  },
  {
    "throughput_mbps": 9.968297910769344,
    "jitter_ms": 1.8626929967072865,
    "loss_percent": 0,
    "distance_m": 25
  },
  {
    "throughput_mbps": 9.97421011568464,
    "jitter_ms": 1.9779903584485938,
    "loss_percent": 0,
    "distance_m": 30
  }
```

Task 4: Include performance_plot.png (or a screenshot of it) and describe the throughput-vs-distance trend you observed

```
osboxes@osboxes:~/netprog-2026/lab08_performance$ source ~/netprog-env/bin/activate
(netprog-env) osboxes@osboxes:~/netprog-2026/lab08_performance$ python3 plot_results.py
Saved performance_plot.png
(netprog-env) osboxes@osboxes:~/netprog-2026/lab08_performance$ deactivate
```



Answer As shown in the graph, when the distance between the station and the access point is increased from 5 to 30 meters, the throughput remains in the range of approximately 9.951–9.974 Mbps consistently close to the target transmission rate of 10 Mbps. While there are minor fluctuations at each measured point, the graph shows a slight increase in throughput by approximately 0.010

Mbps between the 20-meter and 25-meter marks. However, there is no clear downward trend, as the values at each distance remain close to 10 Mbps, with variations occurring only at the decimal level.

Task 5: Describe the loss-vs-distance trend from the same plot, specifically noting whether loss stays near 0% until close to range=30 or increases more gradually.

Answer The packet loss rate remained at 0% across all distances from 5 to 30 meters, with no increase observed even at the 30 meters mark, which corresponds to the maximum range specified for the access point (AP). These results indicate that no packets were lost during the measurement, and no point of increasing packet loss was identified within the tested range.

Task 6: In 2-3 sentences, explain why performance degrades as distance approaches the AP's configured range, and note one-way Mininet-WiFi's simplified distance-based model differs from real-world radiofrequency propagation (e.g., no multipath, obstacles, or interference).

Answer As a station moves further away from an access point (AP), the received signal strength (RSSI) weakens and the signal-to-noise ratio (SNR) decreases. This can lead to reduced throughput and increased packet loss. Near the edge of the coverage range, the station may even lose its connection entirely. So, in this lab, the use of Mininet-WiFi, which employs a simplified distance-based model does not account for factors such as multipath propagation, physical obstacles, or interference, all of which are present in real-world networks.

Verification Checklist

#	Checkpoint	From	✓
1	Manual iperf3 UDP baseline (Part B) produces valid, readable JSON output	Task 3 (Image 2)	☑
2	run_iperf3_udp() (Part C) returns a correct dictionary for a single measurement	Task 2	☑
3	Full distance sweep (Part D) completes and produces a 6-entry results.json	Task 3	☑
4	performance_plot.png (Part E) is generated with both panels correctly labeled	Task 4 (Graph image)	☑
5	Throughput and loss trends are correctly described relative to range=30 (Task 4-5)	Task 4-5 and reflection question	☑

Reflection Questions

Answer briefly (2-4 sentences each) in your report:

1.Why does this lab use iperf3's UDP mode (-u) rather than TCP mode to measure jitter and packet loss? What would you have to do differently with TCP to estimate similar metrics?

Answer TCP employs retransmissions to ensure complete data delivery and utilizes congestion control, which masks packet loss and jitter making direct measurement impossible. In contrast, UDP transmits at a constant rate and provides explicit reports on jitter and loss. When using TCP, one must rely on indirect assessment methods, such as analyzing retransmission data from iperf3 results or measuring RTT and packet loss separately using ping.

2.Why is a short pause (time.sleep) necessary after repositioning a station and before running the next performance measurement?

Answer Because time.sleep allows the system time to simulate and update location data, signal values, and connection status before measurements begin. Taking measurements immediately could result in unstable values or reflect previous locations or transitional states rather than the current, updated conditions.

3.Looking at your plotted results, at what distance does performance begin to degrade noticeably, and how does this relate to the range=30 parameter configured on the AP?

Answer This lab studies the effect of distance from an Access Point on wireless network performance by using Mininet-WiFi to simulate one AP and two stations. The transmitting station is then moved to distances of 5, 10, 15, 20, 25, and 30 meters. Using iperf3 with UDP, data is transmitted at a target rate of 10 Mbps for 5 seconds per distance, and Python is used to record the Throughput, Jitter, and Packet Loss into a JSON file before generating graphs with matplotlib. The results show that the throughput is close to 10 Mbps and the packet loss is 0% at all distances. Performance begins to drop significantly at 15 meters, where throughput decreases slightly, and increases dramatically from 20 meters to 25 meters when vie wed from the slope of the graph. In conclusion, no significant decrease showing a major numerical difference at the non-decimal integer level was found within the 5–30 meter range under this experimental condition.

End of Lab Sheet 8